Planejamento Lógico TaskTracker

Bootcamp II Desafio Cumulativo Fase 1

Identificação	Informação
Estudante	Vinícius Cesar Sena Torres
Curso	Ciência de Dados e Machine Learning
Polo ou turma	Turma B - Asa norte
E-mail institucional	vinicius.cst@sempreceub.com
Disciplina	Bootcamp II
Modalidade	EAD

1 Apresentação e entendimento do problema

1.1 Contexto

Organizar tarefas exige acompanhar o que precisa ser feito, quais atividades são mais urgentes e quais prazos estão próximos. Quando essas informações ficam dispersas em anotações, mensagens ou apenas na memória, o usuário pode esquecer compromissos, perder prazos e executar atividades em uma ordem pouco eficiente.

O TaskTracker é uma solução simples de gerenciamento de tarefas pessoais. Seu propósito é reunir as atividades em um único sistema e permitir que o usuário cadastre, consulte, conclua e remova tarefas. Cada registro terá informações suficientes para identificar a atividade, compreender sua finalidade, reconhecer sua prioridade e acompanhar seu prazo e seu estado atual.

1.2 Problema a ser resolvido

O problema central é a dificuldade de manter uma visão clara das tarefas pendentes e concluídas. O sistema deverá reduzir essa desorganização por meio de um fluxo padronizado: receber os dados, validá-los, armazenar a tarefa e apresentar as informações de maneira legível.

1.3 Público e objetivo

O sistema poderá ser utilizado por estudantes, profissionais e outras pessoas que desejem organizar atividades cotidianas. O objetivo geral é desenvolver, nas próximas fases do Bootcamp, uma aplicação em Python executada inicialmente no terminal, com lógica clara e estrutura preparada para versionamento no GitHub e execução futura em um contêiner Docker.

1.4 Escopo planejado

- Cadastrar uma nova tarefa com título, descrição, prioridade e data-limite.
- Listar todas as tarefas e apresentar seus dados de forma organizada.
- Alterar o status de uma tarefa de pendente para concluída.
- Remover uma tarefa cadastrada mediante confirmação.
- Exibir um resumo com o total de tarefas, pendentes e concluídas.
- Validar os principais dados antes de realizar cada operação.

Nesta primeira versão, as tarefas serão mantidas em memória enquanto o programa estiver em execução. Banco de dados, contas de usuário e interface gráfica não fazem parte do escopo inicial, pois poderão aumentar a complexidade antes da validação da lógica principal.

2 Decomposição do problema e dados do sistema

A decomposição abaixo separa o sistema em dados de entrada, informações de saída e regras. Essa separação facilita a transformação do planejamento em funções Python na Fase 2.

2.1 Entradas de dados

Entrada	Tipo	Finalidade	Exemplo
Opção do menu	Número inteiro	Operação que o usuário deseja executar	1 a 5
Título	Texto	Nome curto que identifica a tarefa	Estudar para a prova
Descrição	Texto	Detalhes sobre a atividade	Revisar os capítulos 1 e 2
Prioridade	Texto	Nível de importância ou urgência	Alta
Data-limite	Data	Prazo para a conclusão	15/09/2026
Identificador	Número inteiro	Código da tarefa a concluir ou remover	3
Confirmação	Texto	Confirma a remoção de uma tarefa	S ou N

2.2 Saídas de dados

Saída	O que será mostrado
Menu principal	Apresenta as operações disponíveis.
Lista de tarefas	Mostra identificador, título, descrição, prioridade, prazo e status.
Mensagem de sucesso	Confirma cadastro, conclusão ou remoção.
Mensagem de erro	Informa dados vazios, opções inválidas ou tarefa inexistente.
Resumo	Mostra os totais de tarefas cadastradas, pendentes e concluídas.

2.3 Representação de uma tarefa

Cada tarefa poderá ser representada em Python por um dicionário com os seguintes campos:

Campo	Tipo	Descrição
id	Inteiro	Identificador único gerado pelo sistema
titulo	Texto	Nome obrigatório da tarefa
descricao	Texto	Detalhamento da atividade
prioridade	Texto	Alta, Média ou Baixa
data_limite	Texto ou data	Prazo informado no formato DD/MM/AAAA
status	Texto	Pendente ou Concluída

2.4 Regras do sistema

Código	Regra	Justificativa
R01	O título é obrigatório e não pode conter apenas espaços.	Evita tarefas sem identificação.
R02	A prioridade deve ser Alta, Média ou Baixa.	Mantém os valores padronizados.
R03	A data-limite deve seguir DD/MM/AAAA e representar uma data válida.	Evita prazos impossíveis ou ilegíveis.
R04	Toda nova tarefa deve receber um identificador único.	Permite localizar a tarefa corretamente.
R05	Toda nova tarefa deve iniciar com o status Pendente.	Padroniza o ciclo de vida da tarefa.
R06	Somente uma tarefa existente e pendente poderá ser marcada como concluída.	Evita alterações inválidas ou repetidas.
R07	Uma tarefa somente será removida após confirmação do usuário.	Reduz exclusões acidentais.
R08	Se a lista estiver vazia, o sistema deverá informar isso claramente.	Evita uma tela vazia sem explicação.
R09	Uma opção inválida não encerrará o programa; o menu será exibido novamente.	Mantém o fluxo controlado.

2.5 Exemplo de funcionamento

Ao cadastrar a tarefa “Finalizar relatório”, com prioridade Alta e data-limite 15/09/2026, o sistema validará os dados, criará um identificador e definirá o status como Pendente. Na listagem, a tarefa aparecerá com todas essas informações. Quando o usuário informar seu identificador na opção de conclusão, o status será alterado para Concluída e o resumo será atualizado.

3 Receita lógica do sistema

O algoritmo foi dividido em pequenos processos para que cada responsabilidade possa se transformar em uma função durante a implementação em Python.

3.1 Fluxo principal

1. Iniciar o programa e criar uma lista vazia para armazenar as tarefas.
2. Exibir o menu com as opções: cadastrar, visualizar, concluir, remover e sair.
3. Solicitar a opção desejada.
4. Verificar se a opção informada é válida.
5. Executar o processo correspondente à opção escolhida.
6. Ao finalizar a operação, mostrar o menu novamente.
7. Repetir o processo até que o usuário escolha sair.
8. Exibir uma mensagem de encerramento e finalizar o programa.

3.2 Cadastro de tarefa

9. Solicitar o título da tarefa.
10. Remover espaços desnecessários e verificar se o título ficou vazio.
11. Se estiver vazio, informar o erro e solicitar o título novamente.
12. Solicitar a descrição da tarefa.
13. Solicitar a prioridade: Alta, Média ou Baixa.
14. Se a prioridade for inválida, mostrar as alternativas e solicitar novamente.
15. Solicitar a data-limite no formato DD/MM/AAAA.
16. Verificar se a data informada possui formato e valor válidos; em caso de erro, solicitar novamente.
17. Gerar um identificador único para a tarefa.
18. Criar a tarefa com os dados informados e status Pendente.
19. Adicionar a tarefa à lista.
20. Exibir a mensagem de cadastro realizado com sucesso.

3.3 Visualização das tarefas

21. Verificar se a lista de tarefas está vazia.
22. Se estiver vazia, informar que não existem tarefas cadastradas e retornar ao menu.
23. Se houver tarefas, percorrer a lista uma tarefa por vez.
24. Exibir o identificador, o título, a descrição, a prioridade, a data-limite e o status de cada tarefa.
25. Contar quantas tarefas estão pendentes e quantas estão concluídas.
26. Mostrar o resumo e retornar ao menu principal.

3.4 Conclusão de tarefa

27. Verificar se existem tarefas cadastradas.
28. Solicitar o identificador da tarefa que será concluída.
29. Procurar na lista uma tarefa com esse identificador.
30. Se a tarefa não existir, exibir uma mensagem de erro.
31. Se a tarefa já estiver concluída, informar que nenhuma alteração é necessária.
32. Se a tarefa existir e estiver pendente, alterar seu status para Concluída.
33. Exibir uma mensagem de sucesso e retornar ao menu.

3.5 Remoção de tarefa

34. Verificar se existem tarefas cadastradas.
35. Solicitar o identificador da tarefa que será removida.
36. Procurar a tarefa correspondente.
37. Se ela não existir, exibir uma mensagem de erro.
38. Se existir, mostrar seu título e solicitar confirmação.
39. Se o usuário confirmar, remover a tarefa e exibir uma mensagem de sucesso.
40. Se o usuário não confirmar, cancelar a operação.
41. Retornar ao menu principal.

3.6 Tratamento de opções inválidas e encerramento

Se o usuário digitar uma opção inexistente, o sistema deverá informar o erro e apresentar novamente o menu, sem perder as tarefas já cadastradas. Quando a opção Sair for escolhida, o sistema exibirá uma mensagem de encerramento e terminará o laço principal.

4 Mapeamento arquitetural para Git e GitHub

O repositório será organizado para separar documentação, código-fonte e configurações. A estrutura inicial planejada é a seguinte:

```
tasktracker-projeto/
|-- README.md
|-- .gitignore
|-- docs/
|   -- planejamento_logico.pdf
-- src/
   -- main.py
```

4.1 Responsabilidade dos arquivos

Arquivo	Responsabilidade
README.md	Apresentará o objetivo, as funcionalidades e as instruções de execução do projeto.
.gitignore	Impedirá o versionamento de arquivos temporários e configurações locais desnecessárias.
docs/planejamento_logico.pdf	Armazenará o planejamento produzido na Fase 1.
src/main.py	Conterá a implementação principal do TaskTracker em Python.

4.2 Relação entre planejamento e implementação

Elemento planejado	Aplicação prevista em Python
Entradas	Comandos input para receber os dados do usuário
Regras	Condições if, elif e else para validar valores
Tarefas	Lista contendo dicionários
Operações	Funções para cadastrar, listar, concluir e remover
Repetição do menu	Laço while até a escolha da opção Sair

5 Link da apresentação em vídeo

O vídeo apresenta o problema resolvido pelo TaskTracker, os dados de entrada e saída, as regras do sistema, o passo a passo do algoritmo e a estrutura planejada para o repositório no GitHub.

Link ativo do vídeo no YouTube: https://youtu.be/bZUXsh_fmsU?si=qk3dbevGdb7O8z8e

Síntese do planejamento

O TaskTracker foi planejado como uma aplicação simples, coerente e possível de implementar em Python. A decomposição dos dados define o que o sistema recebe e apresenta; as regras protegem a consistência das tarefas; o algoritmo descreve cada operação; e a estrutura do repositório prepara a continuidade no GitHub. Assim, o documento funciona como o blueprint que orientará as próximas fases do projeto cumulativo.